

Réseaux de neurones pour l'apprentissage par renforcement dans le jeu de Pacman

Projet: IFT-7025, Automne 2025

Gaétan Allaire

December 21, 2025

Sommaire

1	Introduction	1
2	Méthodes expérimentales	2
2.1	Définition de la représentation du jeu	2
2.2	MLPRegressorQAgent	2
2.3	Hyperparameter tuning	2
3	Evaluation et résultats	3
3.1	Evaluation standard	3
3.2	Evaluation de la généralisation	3
3.3	Evaluation de la régularisation	4
4	Discussion	4
4.1	Performances	4
4.2	Comparaison avec le TP2	4
4.3	Critiques	5
5	Annexes	6
5.1	Résultats des recherches par grille	6
5.2	Résultats de l'évaluation standard	6
5.3	Résultats de l'évaluation de la généralisation	7
5.4	Résultats de l'évaluation de la régularisation	7

1 Introduction

Dans ce projet, nous ajoutons un réseau de neurones perceptron multi-couches (MLPRegressor de scikit-learn) à l'algorithme d'apprentissage par renforcement Pacman, notamment pour l'apprentissage de sa fonction Q . Nous commencerons par définir un nouveau feature extractor spécifique pour le MLPRegressor, puis un nouvel agent basé sur la logique du ApproximateQAgent, qui intégrera le MLPRegressor pour apprendre la fonction Q . Finalement, nous évaluerons notre modèle à travers différents types de tests. Une attention particulière sera portée sur l'étude de l'influence des paramètres du perceptron, notamment la régularisation du modèle, ainsi que sur sa capacité à généraliser son apprentissage.

2 Méthodes expérimentales

2.1 Définition de la représentation du jeu

La nouvelle représentation du jeu est définie dans la classe **MLPRegressorExtractor** comme un feature extractor spécifique au nouvel agent que l'on construira par la suite basé sur le réseau MLP.

On définit une fonction de représentation $\phi(s)$ implémentée dans la classe **MLPRegressorExtractor**, qui extrait un ensemble de caractéristiques pertinentes à partir de l'état courant du jeu et de l'action considérée. Plutôt que d'utiliser une représentation exhaustive de la grille du jeu avec l'ensemble des features (positions des murs, des nourritures et des fantômes), on choisit une représentation plus compacte. Une dimension de l'espace d'entrée trop grande rendrait l'approximation de la fonction **Q** plus difficile et plus instable.

Les 6 features retenues sont:

- Biais**: une constante égale à 1, permettant au réseau de neurones d'apprendre un terme indépendant
- Position normalisée de Pacman**: la coordonnée x de Pacman normalisée par la largeur de la grille et la coordonnée y de Pacman normalisée par la hauteur de la grille. Fournit une information globale sur la progression dans l'environnement
- Nombre de nourritures restantes**: normalisé par la surface totale de la grille. Donne une information sur l'état d'avancement de l'épisode
- Nombre de fantômes à une case de distance**: après l'exécution de l'action pour savoir si un danger est présent. Permet de pénaliser les actions dangereuses susceptibles de mener à une transition terminale négative
- Distance à la nourriture la plus proche**: après l'action, normalisée par la surface de la grille. Indicateur direct de la récompense future, favorisant les actions qui rapprochent Pacman de son objectif principal

Toutes les caractéristiques continues sont normalisées dans l'intervalle $[0,1]$ pour éviter des échelles de valeurs hétérogènes entre les entrées et améliorer la stabilité et la convergence.

2.2 MLPRegressorQAgent

Le **MLPRegressorQAgent** est l'agent qui intègre le **MLPRegressor** de scikit-learn dans l'apprentissage de la fonction **Q**. Il est basé sur l'**ApproximateQAgent**. La méthode *getQ-Value(self, state, action)* appelle la prédiction *predict()* du réseau MLP pour calculer la valeur de **Q**, qui remplace la somme pondérée par les poids de l'agent du TP2. De même, la méthode *update(self, state, action, nextState, reward)* implémente le nouveau feature extractor **MLPRegressorExtractor** pour extraire les caractéristiques de l'état de jeu puis utilise la méthode *fit()* du réseau MLP pour mettre à jour les poids du modèle.

2.3 Hyperparameter tuning

Pour déterminer les paramètres, on implémente une recherche par grille sur les paramètres suivants:

- **hidden_layer_sizes**: la taille des couches cachées, on testera une couche de 64 neurones, une couche de 32 neurones et deux couches de 64 neurones
- **activation**: la fonction d'activation, on testera la ReLU et la sigmoïde
- **solver**: l'optimiseur, on testera Adam et SGD
- **learning_rate_init**: la valeur d'initialisation du taux d'apprentissage, on testera 0.0001 et

0.001

- **warm_start**: le warm_start sera toujours réglé à "True" pour l'appel multiples à la méthode fit

Cela nous donne 24 combinaisons d'hyperparamètres à tester. On effectue trois recherches par grille afin de moyenner les scores. Les entraînements sont fait sur 2000 parties et les tests sur 100 parties, sur la grille smallGrid avec un RandomGhost. Les scores correspondent aux win rates sur les 100 parties de test. Les résultats présents dans la Table 1 permettent d'identifier deux combinaisons d'hyperparamètres préférées:

```
{
  'hidden_layer_sizes': (64, 64),
  'activation': 'logistic',
  'solver': 'sgd',
  'learning_rate_init': 0.001,
  'warm_start': True,
},
{
  'hidden_layer_sizes': (64, 64),
  'activation': 'logistic',
  'solver': 'sgd',
  'learning_rate_init': 0.0001,
  'warm_start': True,
}
```

3 Evaluation et résultats

Pour les évaluations, on effectue plusieurs benchmarks. Le nombre de parties d'entraînement est fixé à 2000 et de test à 100. Le score est définit comme le win rate sur les 100 parties de test.

3.1 Evaluation standard

Le premier test consiste en une évaluation standard du modèle avec les meilleurs hyperparamètres sur différentes grilles, avec les deux types de fantômes. On a donc 4 cas de tests:

- Evaluation on smallGrid with RandomGhost
- Evaluation on mediumGrid with RandomGhost
- Evaluation on smallGrid with DirectionalGhost
- Evaluation on mediumGrid with DirectionalGhost

Les résultats sont présents dans la Table 2.

3.2 Evaluation de la généralisation

Le deuxième consiste en une étude de la capacité du modèle à généraliser. Pour cela, on fait des entraînements dans une certaine configuration puis on test sur une autre. On peut changer la grille ou bien le type de fantôme. On définit les 4 cas de tests suivants:

- Train on smallGrid with RandomGhost, test on mediumGrid with RandomGhost
- Train on mediumGrid with RandomGhost, test on smallGrid with RandomGhost
- Train on smallGrid with RandomGhost, test on smallGrid with DirectionalGhost
- Train on mediumGrid with RandomGhost, test on mediumGrid with DirectionalGhost

Les résultats sont présents dans la Table 3.

3.3 Evaluation de la régularisation

Enfin, on étudie l'influence de la régularisation sur la capacité du modèle à généraliser. Pour cela, on essaie différentes valeurs pour l'hyperparamètre α qui représente le terme de régularisation L2 du modèle. La régularisation ajoute une pénalité sur les poids avec un terme proportionnel (de coefficient α) à la magnitude des poids au carré. Elle permet notamment de réduire le risque de surapprentissage. On test en particulier 5 valeurs de α avec lesquelles on réeffectue les évaluations de généralisation:

alphas = [0.0, 0.0001, 0.001, 0.01, 0.1] (0.0 pas de régularisation)

Les résultats sont présents dans la Table 4.

4 Discussion

4.1 Performances

L'agent utilisant le MLP est globalement très bon sur les grilles plus larges. Il gagne toutes ses parties en mediumGrid quelque soit le type de fantôme. Pour la smallGrid, où les possibilités de mouvements et donc d'échapper au fantôme sont réduites, les scores sont plus faibles (0.74 pour le RandomGhost et faiblit jusqu'à 0.33) (voir Table 2).

Au niveau de la généralisation de la taille de grille, le modèle est très puissant dans l'augmentation de la taille de grille (score de 1.00 lors de l'augmentation de la taille de la grille entre l'entraînement et le test) mais moins performant lors de la réduction (score de 0.81). De plus, la généralisation sur le type de fantôme est très peu performante en smallGrid mais excellent sur mediumGrid (score de 0.21 contre le DirectionalGhost après un entraînement sur le RandomGhost sur smallGrid et 1.00 sur mediumGrid) (voir Table 3).

Les résultats (voir Table 4 et Table 5) montrent que la régularisation n'améliore pas la capacité de généralisation de l'agent. Au contraire, une augmentation du coefficient α entraîne une dégradation progressive des performances, suggérant que le modèle ne souffrait pas initialement de surapprentissage. La difficulté principale réside davantage dans le changement de distribution induit par le type de fantôme que dans la complexité du réseau.

4.2 Comparaison avec le TP2

L'agent de Q learning approximatif implémenté dans le TP2 semble être plus performant sur smallGrid peu importe le fantôme (score de 1.00 dans les deux cas) mais est très peu performant sur une grille plus large (score de 0.09 et 0.02 sur mediumGrid) (voir Table 2). Son utilisation est donc meilleure dans des situations plus simples, mais un problème plus complexe nécessiterait d'adopter un modèle d'agent plus complexe comme le notre utilisant un perceptron multi-couches. Globalement, l'agent basé sur MLP bénéficie plus d'un plus grand espace de recherche et des situations plus complexes.

4.3 Critiques

Finalement, on a un modèle qui est plus adapté pour des situations de jeux complexes. En revanche, pour des problèmes simples, la complexité du modèle est inutile et même cause d'une mauvaise performance. Une grande partie des propriétés du modèle vient sans doute de la définition des features utilisées dans l'extracteur. Une étude plus approfondie pourrait fournir des détails sur l'impact des différentes features et ainsi ajouter de nouvelles features plus déterminantes ou retirer celles qui ne causeraient qu'une augmentation de dimension inutile.

Le modèle par défaut ne semble pas souffrir de surapprentissage. Il pourrait être intéressant d'analyser plus précisément le nombre de parties d'entraînement nécessaires. En effet, tous les entraînements ont été fait sur 2000 parties, mais le modèle apprend à chaque appel de méthode *fit()* du réseau MLP, dont le nombre dépend du nombre d'action dans la partie. La quantité d'apprentissage peut donc varier selon les parties et particulièrement selon la taille de la grille (un modèle entraîné sur `mediumGrid` apprendra plus en moyenne qu'un modèle entraîné sur `smallGrid`).

Enfin, l'optimisation des hyperparamètres a été faite entièrement sur `smallGrid` avec `RandomGhost`, mais il serait plus optimal de la refaire pour les cas `mediumGrid` et `DirectionalGhost`.

5 Annexes

5.1 Résultats des recherches par grille

Couches cachées	Activation	Optimiseur	LR Init	Score 1	Score 2	Score 3
(64,)	ReLU	Adam	.0001	.40	.33	.43
(32,)	ReLU	Adam	.0001	.42	.39	.40
(64, 64)	ReLU	Adam	.0001	.39	.39	.36
(64,)	Sigmoïde	Adam	.0001	.30	.47	.40
(32,)	Sigmoïde	Adam	.0001	.39	.42	.43
(64, 64)	Sigmoïde	Adam	.0001	.58	.61	.61
(64,)	ReLU	SGD	.0001	.78	.77	.68
(32,)	ReLU	SGD	.0001	.75	.71	.68
(64, 64)	ReLU	SGD	.0001	.00	.00	.00
(64,)	Sigmoïde	SGD	.0001	.73	.73	.79
(32,)	Sigmoïde	SGD	.0001	.73	.78	.74
(64, 64)	Sigmoïde	SGD	.0001	.82	.80	.77
(64,)	ReLU	Adam	.001	.41	.34	.39
(32,)	ReLU	Adam	.001	.34	.43	.40
(64, 64)	ReLU	Adam	.001	.41	.34	.35
(64,)	Sigmoïde	Adam	.001	.42	.29	.39
(32,)	Sigmoïde	Adam	.001	.47	.35	.39
(64, 64)	Sigmoïde	Adam	.001	.39	.38	.35
(64,)	ReLU	SGD	.001	.00	.00	.00
(32,)	ReLU	SGD	.001	.00	.00	.00
(64, 64)	ReLU	SGD	.001	.00	.00	.00
(64,)	Sigmoïde	SGD	.001	.69	.68	.70
(32,)	Sigmoïde	SGD	.001	.75	.70	.69
(64, 64)	Sigmoïde	SGD	.001	.95	.77	.71

Table 1: Résultats des recherches par grille

5.2 Résultats de l'évaluation standard

Grille	Fantôme	Score	Score (TP2)
smallGrid	RandomGhost	.74	1.00
smallGrid	DirectionalGhost	.33	1.00
mediumGrid	RandomGhost	1.00	.09
mediumGrid	DirectionalGhost	1.00	.02

Table 2: Résultats de l'évaluation standard

5.3 Résultats de l'évaluation de la généralisation

Grille d'entraînement	Grille de test	Fantôme d'entraînement	Fantôme de test	Score
smallGrid	mediumGrid	RandomGhost	RandomGhost	1.00
mediumGrid	smallGrid	RandomGhost	RandomGhost	.81
smallGrid	smallGrid	RandomGhost	DirectionalGhost	.21
mediumGrid	mediumGrid	RandomGhost	DirectionalGhost	1.00

Table 3: Résultats de l'évaluation de généralisation

5.4 Résultats de l'évaluation de la régularisation

α	Résultats				
	Grille d'entraînement	Grille de test	Fantôme d'entraînement	Fantôme de test	Score
0.0	smallGrid	mediumGrid	RandomGhost	RandomGhost	1.00
	mediumGrid	smallGrid	RandomGhost	RandomGhost	.81
	smallGrid	smallGrid	RandomGhost	DirectionalGhost	.21
	mediumGrid	mediumGrid	RandomGhost	DirectionalGhost	1.00
0.0001	smallGrid	mediumGrid	RandomGhost	RandomGhost	1.00
	mediumGrid	smallGrid	RandomGhost	RandomGhost	.48
	smallGrid	smallGrid	RandomGhost	DirectionalGhost	.19
	mediumGrid	mediumGrid	RandomGhost	DirectionalGhost	.95
0.001	smallGrid	mediumGrid	RandomGhost	RandomGhost	1.00
	mediumGrid	smallGrid	RandomGhost	RandomGhost	.65
	smallGrid	smallGrid	RandomGhost	DirectionalGhost	.15
	mediumGrid	mediumGrid	RandomGhost	DirectionalGhost	.95
0.01	smallGrid	mediumGrid	RandomGhost	RandomGhost	.40
	mediumGrid	smallGrid	RandomGhost	RandomGhost	.76
	smallGrid	smallGrid	RandomGhost	DirectionalGhost	.18
	mediumGrid	mediumGrid	RandomGhost	DirectionalGhost	.81
0.1	smallGrid	mediumGrid	RandomGhost	RandomGhost	1.00
	mediumGrid	smallGrid	RandomGhost	RandomGhost	.54
	smallGrid	smallGrid	RandomGhost	DirectionalGhost	.19
	mediumGrid	mediumGrid	RandomGhost	DirectionalGhost	.53

Table 4: Résultats de l'évaluation de la régularisation

α	Score moyen
0.0	.76
0.0001	.66
0.001	.69
0.01	.54
0.1	.57

Table 5: Résultats moyens de l'évaluation de la régularisation